Towards a Digital Twin of Ottawa's Transportation System

Progress Report

Braeden Kloke

Renee Hogarth-Rosmus

Sebastian Ezman

Group 34

SYSC 4907A: Engineering Project

Department of Systems and Computer Engineering

Supervisor: Professor Gabriel Wainer

December 5, 2025

ABSTRACT

This project's objective is to design and implement the foundation for a Digital Twin of Ottawa's transportation system. We aim to demonstrate the system through a model and simulation of a university campus evacuation. This progress report describes our design, modifications from the initial proposal, and a revised project timeline.

TABLE OF CONTENTS

1. INTRODUCTION 5
2. BACKGROUND 5
2.1. DIGITAL TWINS 5
2.2. OPEN STREET MAP 6
2.3. DIFFUSION ABSTRACT MODEL (DAM) 6
2.4. DISCRETE EVENT SYSTEM SPECIFICATION (DEVS) 7
3. SYSTEM DESIGN 7
3.1. REQUIREMENTS 7
3.2. ARCHITECTURE 8
3.3. CONFIGURATION 9
3.4. PRESENTATION LAYER 10
4. CASE STUDY: UNIVERSITY CAMPUS EVACUATION 10
4.1. EXPERIMENTAL FRAME 10
4.2. IMPLEMENTATION AND SIMULATION 10
4.3. TESTING AND VALIDATION 11
5. CONCLUSION 11
ACKNOWLEDGEMENTS 11
APPENDIX A: DEVS MODELS *12*
REFERENCES *13*

TABLE OF FIGURES

Figure 1: DAM without Indirect Links. 6
Figure 2: System DAM. 8
Figure 3: DEVS coupled model for an Intersection. 9
Figure 4: Data Pipeline for configuring the DEVS atomic models. 9
Figure 5: Gantt chart of revised project timeline. 11

LIST OF TABLES

Table 1: Functional requirements. 7
Table 2: Non-functional requirements. 8
Table 3: List of intersections, roads and parking lots to model. 10
Table 4: *Intersection* atomic DEVS model. 12
Table 5: *Road* atomic DEVS model. 13
Table 6: *Parking Lot* atomic DEVS model. 13

1. INTRODUCTION

The project's objective remains the same as described in [1]: to develop the foundation for a Digital Twin (DT) of Ottawa's transportation system. This foundation is a Digital Model that is configurable with real-world geographic data and can simulate urban mobility. Considerations are made for how the system can be evolved into a Digital Shadow and, ultimately, a DT. We plan to demonstrate the system with a case study of a university campus evacuation.

Throughout the project we followed the Agile methodology to develop rapid prototypes that were continuously iterated on. We have completed the modeling phase for the timeline proposed in [1] and are well into the implementation phase.

The rest of this report is organized as follows. Section 2 discusses relevant background knowledge to our project. Section 3 describes the current state of our design and modifications from the original proposal. Section 4 describes the case study for university campus evacuation. Lastly, Section 5 concludes the report and presents a revised project timeline.

2. BACKGROUND

2.1. DIGITAL TWINS

DTs are composed of three key elements: a physical entity, a digital model representing that entity, and a data flow that enables communication between the two of them [2]. A fully realized DT maintains continuous synchronization between the digital and physical systems and possesses the capability to control physical assets. DTs are a technology for the analysis and optimization of complex systems by providing real-time monitoring, simulation of 'what-if' scenarios, and decision support. Their ability to twin an urban environment can help to improve travel times, adapt to dynamic traffic demands, and reduce operation costs and carbon-dioxide emissions.

DTs can be classified into a hierarchy based on the level of integration between physical and digital components [3]. A Digital Model operates independently of its physical counterpart with its data updated asynchronously, thus it is not a live, up-to-date copy of the physical entity. A Digital Shadow is continuously updated with real-world data and is a current representation of the physical entity. Lastly, a DT, the most advanced form, involves two-way communication between the digital and physical entity, such that the DT can influence the state of the physical asset.

To illustrate the distinctions, consider a bus transportation system. A Digital Model might use historical bus route data to simulate traffic flow. A Digital Shadow would continually update the model with GPS data from real-world buses, such that the model would be a current representation of the physical system. Lastly, a DT could monitor the busses in real-time, make predictions, and reallocate buses based on traffic and demand.

2.2. OPEN STREET MAP

OpenStreetMap (OSM) is a collaborative, open access geographic database that can provide detailed information on roads, intersections, buildings, and map features. The data is collected through a combination of community contributions, GPS traces, and satellite imagery. OSM has its data structured using a network of nodes, ways and relations. This structured geographical data is essential for developing a DT because it provides an accurate, real-world representation of the physical environment. Using OSM data will allow the project to be automatically populated with accurate road geometrics, and intersection layouts enabling the simulation modeling to mirror realistic behaviours.

2.3. DIFFUSION ABSTRACT MODEL (DAM)

An object spreading throughout an environment from an area of high concentration to an area of low concentration, like a sugar cube dissolving in a glass of hot tea, is known as a *diffusion process*. Urban mobility can be viewed as the diffusion process of people throughout a city. For example, the evacuation of a university campus would be the spread of people moving from the highly concentrated campus to lower concentration areas throughout the city.

The Diffusion Abstract Model (DAM) is used to model and simulate diffusion processes [4]. Figure 1 shows the DAM. The *Diffusion Element Generator* creates the diffusion elements that are spreading throughout the system. These generators then transmit the diffusion elements to a network node. A *Node* is an agent-based model which decides how to handle the spread of the diffusion element throughout the system. A *Direct Link* connects nodes together to create the network structure. Lastly, *Node Updaters* and *Direct Link Updaters* enable dynamic changes to the network. The *Indirect Links* are omitted since they are not utilized in this project

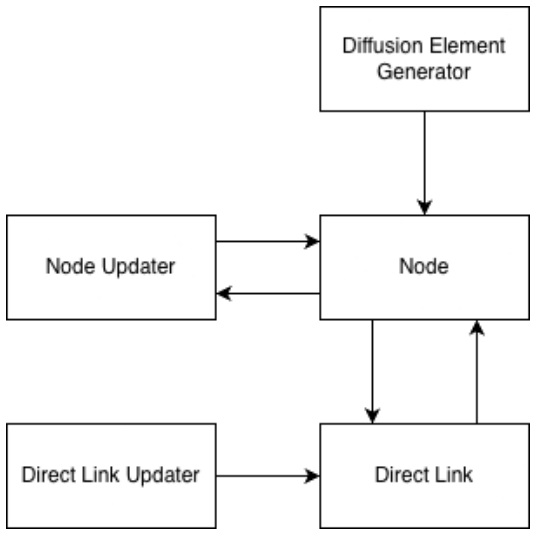

Figure 1: DAM without Indirect Links.

2.4. DISCRETE EVENT SYSTEM SPECIFICATION (DEVS)

DEVS [5] is a formalism for modeling and simulating discrete event systems. An atomic DEVS model is defined as:

$$M = \langle X, Y, S, \delta_{ext}, \delta_{int}, \lambda, ta \rangle$$

S is the set of states in the model. State transitions can be time-based or based on the reaction to external events. $\delta_{int}: S \rightarrow S$ is the internal transition function which causes the model to change state after a certain amount of time. $ta: S \rightarrow \mathbb{R}^{+}_{0,+\infty}$ is the time advance function and determines when the next internal state transition will occur, a return value of infinity means the model is in a *passive* state and will remain in that state indefinitely unless prompted with input. *X* is the set of inputs the model can react to. $\delta_{ext}: S \times e \times X \rightarrow S$ is the external transition function that may cause the model to change state when it receives an input, e is the time elapsed in the model's current state. *Y* is the set of outputs the model can produce. $\lambda: S \rightarrow Y$ is the output function.

Additionally, atomic DEVS models can be coupled together to form hierarchies of DEVS models with any level of abstraction.

3. SYSTEM DESIGN

This section outlines the current state of our system's design.

3.1. REQUIREMENTS

Table 1 and Table 2 list the updated requirements from [1]. We've refined our functional requirements to emphasize a Digital Model that is extendable into a DT. FR5 and FR6 support the evolution of the tool in a DT, though they will not be implemented in this iteration of the project. The non-functional requirements have been reworded but are semantically the same.

Table 1: Functional requirements.

ID	Functional requirement
FR1	The system shall model and simulate the flow of vehicles in an urban environment.
FR2	System models shall be configurable with geographic data.
FR3	Simulation results shall be output in GeoJSON format.
FR4	The system shall have a presentation layer for visualizing simulations.
FR5*	The system shall process input from external systems during runtime, e.g., input from the Smart Open Hamburg project.
FR6*	The system shall be able to control physical assets.

Table 2: Non-functional requirements.

ID	Non-functional requirements
NFR1	System models shall be designed for reuse.
NFR2	System inputs shall preserve the privacy of citizens.

3.2. ARCHITECTURE

We've chosen the DAM to be the architecture of our Digital Model. We evaluated this against the Agents, Entities, Layers architecture of the Smart Open Hamburg system [6]. Figure 2 shows the DAM. *Vehicles* are our diffusion elements, our object that spreads throughout the system. *Intersections* are our nodes that control the spread of vehicles throughout the system. The agent behaviour of the Intersections is configured using origin-destination (OD) data. Intersections are directly linked by *Roads*. Lastly, *Intersection Updater* and *Road Updater* provide dynamic updates to Intersections and Roads respectively. *Parking Lots* are our diffusion element generators which create and transmit vehicles to Intersections.

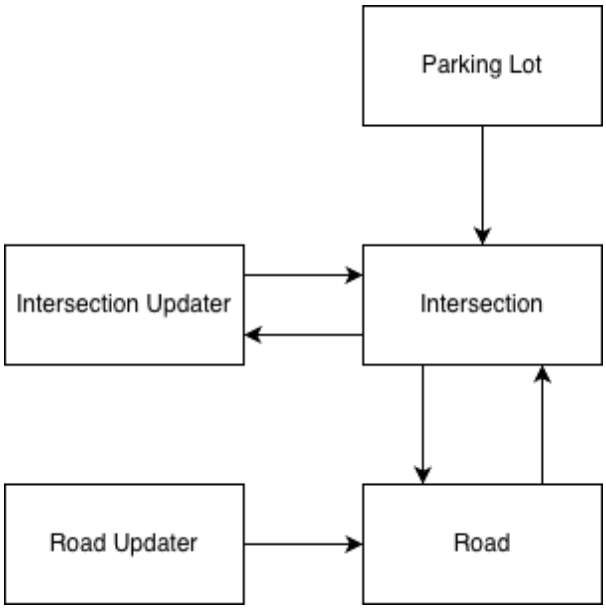

Figure 2: System DAM.

We've chosen to express the DAM using the DEVS formalism, such that the *Road, Intersection* and *Parking Lot* each correspond to an atomic DEVS model. Appendix A contains the formal definitions of these DEVS models.

To illustrate how these models are coupled together, Figure 3 shows a DEVS coupled model of a generic *Intersection A* which has inputs of vehicles from *Road A*, *Road B*, and *P1*. Upon receiving an input, *Intersection A* selects a destination based on its OD data, then routes the vehicle to an output port, either Road C or Road D. Output ports *out3* and *out4* are left inactive.

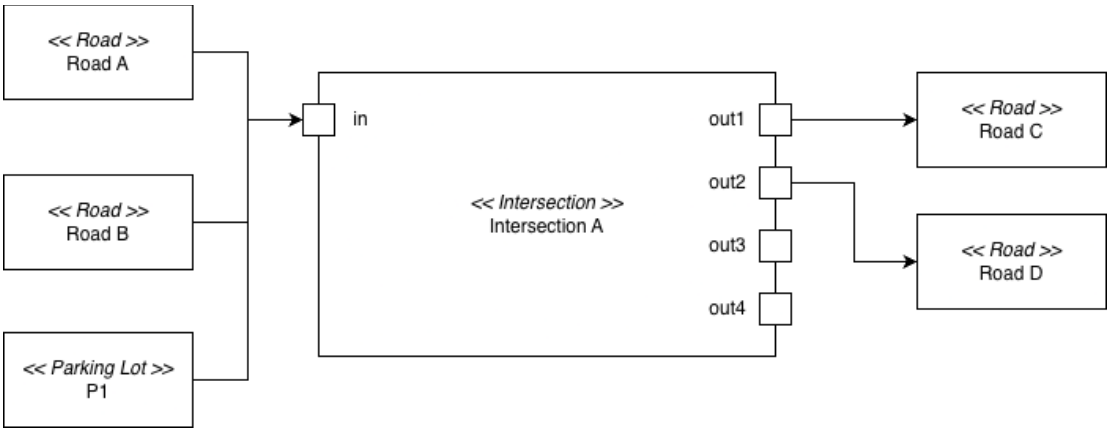

Figure 3: DEVS coupled model for an Intersection.

3.3. CONFIGURATION

DEVS atomic models are configured through the data pipeline outlined in Figure 4. OSM data is first extracted using a python script that incorporates the OSMX library to produce structured road, and intersection information. This processed data is then mapped into the corresponding DEVS atomic model, which are ultimately assembled within the top-level coupled model.

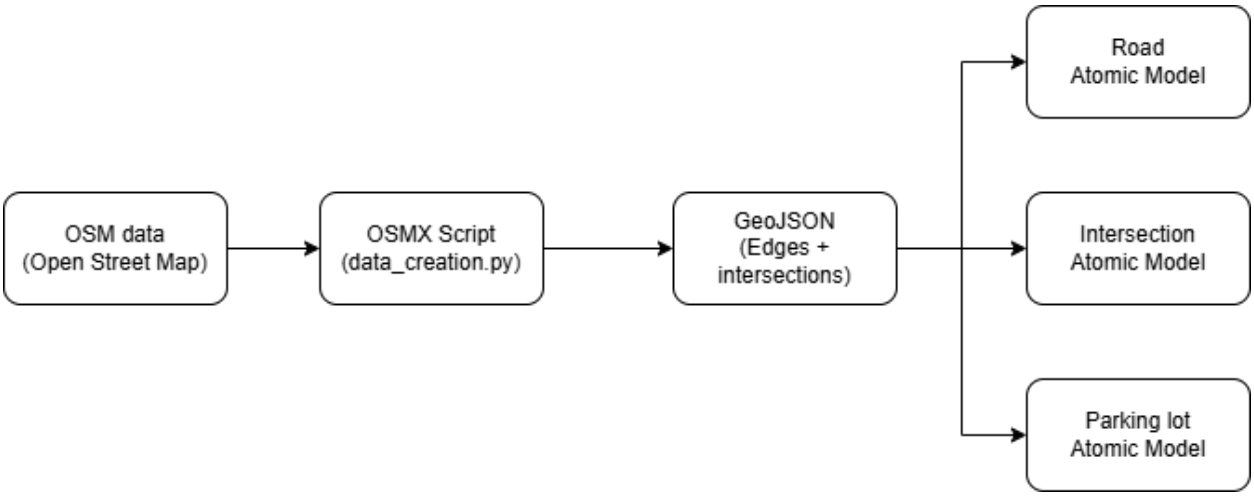

Figure 4: Data Pipeline for configuring the DEVS atomic models.

The agent behaviour of the Intersections is configured using OD data, that is, Intersections will make decisions on how to route traffic based on the flow rate from the OD data. We plan to synthetically generate this OD data.

3.4. PRESENTATION LAYER

Simulation outputs will be formatted as GeoJSON to be input into a presentation layer. GeoJSON was selected to facilitate interoperation with other systems such as Kepler [7].

4. CASE STUDY: UNIVERSITY CAMPUS EVACUATION

We've chosen to demonstrate our system by simulating an evacuation of Carleton University's campus.

4.1. EXPERIMENTAL FRAME

Our experimental frame for our study is a university campus evacuation, where we will study the spread of *Vehicles* through Carleton University's transportation network of *Roads*, *Intersections*, and *Parking Lots*. We've simplified our study to not account for pedestrians, bicycles, buses or trains. We also assume that vehicles are only generated from parking lots. Table 3 lists the objects to model for our case study.

Table 3: List of intersections, roads and parking lots to model.

Intersections	**Roads**	**Parking Lots**
Campus Ave & Library Road	Campus Ave	P1
Campus Ave & P6	Library Rd	P3
Campus Ave & University Dr	Raven Rd	P4
Library Rd & P1	Stadium Way	P5
Library Rd & University Dr	University Dr	P6
P3 & Raven Rd		P7
P4 & University Dr		
P5 & Stadium Way		
Raven Rd & University Dr		
Stadium Way & University Dr		
University Dr Roundabout		

4.2. IMPLEMENTATION AND SIMULATION

We've chosen the Cadmium tool [8] to be the simulation tool for our system. Again, this tool was evaluated against the MARS platform for the Smart Open Hamburg project.

We will use OpenStreetMap data to configure the network structure and we will synthetically create the OD data to configure the agent behaviour of the Intersections.

We plan to use Kepler [7], an open-source geospatial analysis tool for large-scale data sets, to visualize the results of our simulation.

4.3. TESTING AND VALIDATION

We plan to write system tests to test the reachability of all Intersections and Roads. These tests should supply the system with inputs of OD data and parking lot schedules that cause each Intersection and Road to be traversed at least once.

We will validate our model with the mentioned reachability tests as well as visual inspection of the presentation layer to see that the model conforms with the real-world.

5. CONCLUSION

This progress report has described the status of our project *Towards a Digital Twin of Ottawa's Transportation System.* Deviations from [1] have been noted throughout the document. Figure 5 shows a Gantt chart for the revised project timeline. An estimated two weeks of further implementation on the simulation models is needed after returning from winter break followed by a month of testing and validation. Documentation milestones have been included for context.

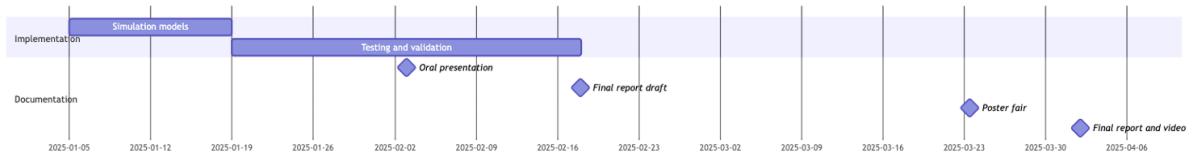

Figure 5: Gantt chart of revised project timeline.

ACKNOWLEDGEMENTS

We would like to thank Dr. Gabriel Wainer for his guidance and supervision throughout the project and to members of Dr. Wainer's Advanced Real-time Simulation (ARS) Lab for helping ideate and evaluate alternatives to our design.

AI was not used to generate code or text for this project.

APPENDIX A: DEVS MODELS

This appendix defines the DEVS atomic models and data structures for our system.

Vehicles $\boldsymbol{V}$ is defined as the set of vehicles:

$$V = \{v_0, v_i, \dots, v_i\}, i = 1, \dots, n$$

Each vehicle is defined as a tuple $v_i = (id, dest, t)$, where:

- *id* is the unique identifier for this vehicle.
- *dest* is the destination for this vehicle.
- *t* is the time in seconds that the vehicle is remaining on the road.

Origin-destination data ***ODData*** is defined as the set of origin-destination datums:

$$ODData = \{od_0, od_i, \dots, od_n\}, i = 1, \dots, n$$

Each origin-destination datum is defined as a tuple $od_i = (origin, dest, flowRate)$, where:

- *origin* is the intersection the vehicle has entered.
- *dest* is the next destination the vehicle is going to.
- *flowRate* is the number of vehicles going from the origin to the destination over a certain amount of time.

Lastly, the output port map $\boldsymbol{OutPortMap}$ is a map of destinations to output ports.

Table 4, Table 5, and Table 6 define the DEVS atomic models for our system.

Table 4: *Intersection* atomic DEVS model.

Parameter	Value
$X =$	$\{(in, \boldsymbol{V})\}$
$Y =$	$\{(out1, \boldsymbol{V}); (out2, \boldsymbol{V}); (out3, \boldsymbol{V}); (out4, \boldsymbol{V})\}$
$S =$	$\{v \in \boldsymbol{V}, od \in \boldsymbol{ODData}, opm \in \boldsymbol{OutPortMap}\}$
$\delta_{ext}(S, e, X) =$	$\{$
	$v \leftarrow in.value;$
	$v.dest \leftarrow selectDest(v, od);$
	$\}$
$\delta_{int}(S) =$	$\{v \leftarrow null\}$
$\lambda(S) =$	$\{$
	$o \leftarrow getOutPort(v.dest, opm);$
	$\boldsymbol{if}\ o = out1\ \boldsymbol{then}\ out1.value \leftarrow v;$
	$\boldsymbol{elif}\ o = out2\ \boldsymbol{then}\ out2.value \leftarrow v;$
	$\boldsymbol{elif}\ o = out3\ \boldsymbol{then}\ out3.value \leftarrow v;$
	$\boldsymbol{elif}\ o = out4\ \boldsymbol{then}\ out4.value \leftarrow v;$
	$\}$
$ta(S) =$	$\{\ \boldsymbol{if}\ v \neq null \rightarrow 0;\ \boldsymbol{else} \rightarrow \infty\}$

Table 5: *Road* atomic DEVS model.

Parameter	Value	
$X =$	$\{(in, \boldsymbol{V})\}$	
$Y =$	$\{(out, \boldsymbol{V})\}$	
$S =$	$\{length \in \mathbb{R}^+, speedLimit \in \mathbb{N}, queue \in \{v \in \boldsymbol{V}\}^*\}$	
$\delta_{ext}(S, e, X) =$	$\{$	
	$\boldsymbol{for}\ v \in queue\ \boldsymbol{do}\ v.t \leftarrow v.t - e\ \boldsymbol{end};$	// Update times of vehicles on // road based on the elapsed time.
	$n \leftarrow in.value;$	
	$n.t \leftarrow calcTimeOnRoad(n, length, speedLimit);$	
	$queue.add(n);$	
	$\}$	
$\delta_{int}(S) =$	$\{$	
	$x \leftarrow queue.pop();$	
	$\boldsymbol{for}\ v \in queue\ \boldsymbol{do}\ v.t \leftarrow v.t - x.t\ \boldsymbol{end};$	
	$\}$	
$\lambda(S) =$	$\{ out.value \leftarrow queue.first(); \}$	
$ta(S) =$	$\{ \boldsymbol{if}\ queue.empty() \rightarrow \infty;\ \boldsymbol{else} \rightarrow queue.first().t \}$	

Table 6: *Parking Lot* atomic DEVS model.

Parameter	Value	
$X =$	$\emptyset$	
$Y =$	$\{(out, \boldsymbol{V})\}$	
$S =$	$\{schedule \in \{r \in \mathbb{R}^+\}^*\}$	// Ordered list of vehicle // departure times.
$\delta_{ext}(S, e, X) =$	$\emptyset$	
$\delta_{int}(S) =$	$\{ schedule.removeFirst(); \}$	
$\lambda(S) =$	$\{ out.value \leftarrow new\ Vehicle(); \}$	
$ta(S) =$	$\{ \boldsymbol{if}\ schedule.empty() \rightarrow \infty;\ \boldsymbol{else} \rightarrow schedule.first() \}$	

REFERENCES

[1] B. Kloke, R. Hogath-Rosmus and S. Ezman, "Towards a Digital Twin of Ottawa's Transportation System," 2025.

[2] C. Ruiz-Martin, G. Wainer and A. Lopez-Paredes, "Discrete-Event Modeling and Simulation of Diffusion Processes in Multiplex Networks," *ACM Transactions on Modeling and Computer Simulation (TOMACS),* vol. 31, no. 1, pp. 1-32, 2020.

[3] B. P. Ziegler, A. Muzy and E. Kofman, Theory of modeling and simulation: discrete event & iterative system computational foundations, Academic press, 2018.

[4] T. Clemen, N. Ahmady-Moghaddam, U. A. Lenfers, F. Ocker, D. Osterholz, J. Strobele and D. Glake, "Multi-Agent Systems and Digital Twins for Smarter Cities," in *Proceedings of the 2021 ACM SIGSIM conference on principles of advanced discrete simulation*, 2021.

[5] OpenJS Foundation, "Kepler | Large-scale WebGL-powered Geospatial Data Visualization Tool," [Online]. Available: https://kepler.gl/.

[6] L. Belloli, D. Vicino, C. Ruiz-Martin and G. Wainer, "Building Devs Models with the Cadmium Tool," in *2019 Winter Simulation Conference (WSC)*, National Harbor, 2020.